Phase 1 Report: Dataset Collection and Tokenizer Construction

Model H: Hindi | Model L: Nepali

Swaraj Kumar
2025202016

August 21, 2026

Abstract

This report covers Phase 1 of the Language Models and Agents individual project: dataset collection, cleaning, splitting, and tokenizer construction for two independent, from-scratch decoder-only Transformer language models. Model H (higher-resource) is **Hindi**; Model L (lower-resource, from the assignment's allowed list) is **Nepali**. Every number in this report is read directly from a file produced by the project's own pipeline code – Hindi from `clean_stats.json` and `hindi_pipeline_manifest.json`; Nepali from `nepali/data/final/corpus_stats.json` and `split_stats.json` – or the trained tokenizer model files re-measured directly, never estimated. Both languages clear the assignment's hard 20% manual- collection floor; Hindi reaches 101.2% of the 500M-token planning target and Nepali reaches 127.0%. Hindi and Nepali run on two structurally different pipelines (Hindi: a single-pass cleaner; Nepali: a stage-based S00–S14 pipeline with MinHash/LSH dedup and absolute per-lane token targets), described separately throughout rather than forced into one shared schema. Nepali is this project's second Model L candidate – Bhojpuri was the original choice, later superseded once Nepali's own pipeline finished; Bhojpuri's tracked pipeline code and tokenizer files have since been removed from the repository, with its measured numbers kept here as the historical record (Section 6).

Contents

1 Overview **2**

2 Data Collection **2**
2.1 Hindi (Model H) raw sources 2
2.2 Nepali (Model L) raw sources 3

3 Cleaning Pipeline **3**
3.1 Hindi cleaning pipeline 3
3.2 Nepali cleaning pipeline 4
3.3 Cross-language check (current: Hindi vs. Nepali) 5

4 Splits **7**

5 Tokenizers **8**
5.1 Hindi: SentencePiece Unigram 8
5.2 Nepali: SentencePiece BPE 8
5.3 Full sweep table 8

5.4 Why UNK rate is meaningless for Hindi but real for Nepali 12
5.5 Vocab-size choice 12
5.6 Sample tokenizations (real model output) 12
5.7 Training sample size – stated plainly 14

6 Results **14**

7 Reproducibility **14**

8 Limitations and What I Would Do Differently **16**

Sources and References **17**

1 Overview

The assignment requires two independent, from-scratch decoder-only Transformer language models in two different Indian languages: Model H from a higher-resource language, Model L from a fixed list of lower-resource languages (Assamese, Bhojpuri, Bodo, Dogri, Konkani, Maithili, Manipuri, Mizo, Nepali, Sindhi). Both must be built with fully separate data, tokenizer, vocabulary, and (from Phase 2) weights – no sharing, no multilingual training.

Model H is Hindi. Hindi is one of the highest-resource Indian languages for NLP: Wikipedia, curated corpora like AI4Bharat's Sangraha, and web-crawled corpora like Fineweb-2 all have large Hindi partitions. This makes it realistic to reach the assignment's ∼500M-token planning target almost entirely from public sources, while still requiring genuine manual collection to clear the 20% floor.

Model L is Nepali. Nepali has considerably more public data available than several other allowed languages – AI4Bharat's Sangraha and IndicCorpV2 both carry sizeable Nepali partitions – which let this corpus not just clear the 20% manual floor but exceed the 500M-token planning target outright (127.0%), while still requiring genuine, sustained manual collection (mainly news-site scraping) to hit the pipeline's own 100M-token absolute manual-lane floor.

Both languages are Devanagari-script. Hindi's cleaning/tokenization code is `scripts/clean_lib.py`; Nepali's is its own independent `corpuslib` package under `nepali/scripts/` – not shared with Hindi's – keeping every byte of data, every trained model file, and every decision fully separate per language.

2 Data Collection

What counts as "manual" under the course ruling

A source counts as **manual** only if it required work beyond pointing a downloader at an existing public dataset: a self-run scrape, a personally extracted Wikipedia XML dump, OCR from a physical or scanned source, or similarly hands-on collection. A dataset that is just *labeled* as Wikipedia-derived on Hugging Face does not count.

2.1 Hindi (Model H) raw sources

	Docs	Characters
Manual	229,011	540,367,035
Downloaded	544,595	1,299,990,112
Total raw	**773,606**	**1,840,357,147**

Table 1: Hindi raw corpus totals.

Manual sources: `hi_wikipedia_dump` (self-extracted Wikipedia XML dump, 160,591 docs / 234.1M chars), `gadya_kosh` (prose wiki, 21,481 / 197.5M), `patrika_hindi` (newspaper archive, 16,251 / 73.8M), `kavita_kosh` (poetry wiki, 30,621 / 23.6M), 67 misc. `archive.org` items (11.4M chars).

Downloaded sources: `HuggingFaceFW/fineweb-2:hin_Deva` (217,460 / 500.0M chars), `ai4bharat/sangraha:unverified/hin` (191,987 / 500.0M), `ai4bharat/sangraha:verified/hin` (web+pdf+ speech, 135,148 / 299.99M).

Excluded: `wikimedia/wikipedia:20231101.hi` (163,091 docs) – quarantined, redundant with the self-extracted dump.

2.2 Nepali (Model L) raw sources

Nepali's pipeline tracks docs/words/tokens per source, not characters – its cleaner never kept a per-lane character tally past normalisation. Two lanes, each gated on its own **absolute** token target rather than a ratio: manual $\geq$ 100,000,000 tokens, downloaded $\geq$ 400,000,000 tokens (`nepali/configs/language.yaml`).

	Docs	Words	Tokens
Manual	337,783	116,207,907	167,787,251
Downloaded	1,580,482	319,077,639	467,417,082
Total (final)	**1,918,265**	**435,285,546**	**635,204,333**

Table 2: Nepali final corpus totals.

Manual sources: `news_onlinekhabar` (web scrape, 308,139 docs, 153.7M tokens – 91.6% of manual tokens), `wikipedia_api` (self-parsed XML dump, 8.1M tokens), `news_bbc_nepali` (5.3M), `wikibooks_api`, `wiktionary_api`, `news_ekantipur`, `news_ratopati`, `ocr_archive_books`, `news_nagarik` (remainder, combined <1M tokens).

`news_onlinekhabar`'s dominance is a real, measured constraint: it is the manual source with the largest scrapable archive, and `nepali/scripts/auto_grow_and_tokenize.py`'s growth loop raised its collection target specifically because the other manual sources are genuinely smaller pools for Nepali than their Hindi equivalents.

Downloaded sources: `ai4bharat/sangraha:verified` (579,743 docs, 258.5M tokens), `ai4bharat/sangraha:unverified` (341,901 docs, 141.9M tokens), `ai4bharat/IndicCorpV2` (658,838 docs, 67.1M tokens).

Growth mechanism. Rather than collecting once and hoping the target was hit, `auto_grow_and_tokenize.py` ran the full S02–S14 pipeline, checked both absolute lane targets against the real measured output, and – for whichever lane fell short – raised that lane's still-growable sources' collection targets and re-ran, up to 4 rounds, with a per-source growth ceiling. Both targets were met without ever loosening a cleaning filter to manufacture volume.

3 Cleaning Pipeline

Hindi and Nepali use two different cleaning implementations, described separately below.

3.1 Hindi cleaning pipeline

`scripts/clean_lib.py`, run via `clean_hindi.py`. Measured stage order, confirmed from `clean_stats.json`'s own checkpoints:

raw → malformed-Unicode removal → too-short removal → language-ID filter → exact/near-duplicate removal → cleaned

Normalization: NFC, not NFKC. NFKC would strip or alter U+200C (ZWNJ) and U+200D (ZWJ) – zero-width joiner/non-joiner characters that are **semantically meaningful**

in Devanagari, controlling whether a consonant cluster forms a conjunct ligature or stays visually separate. NFC preserves them.

Latin-token removal, digits kept. 8,016,102 residual Latin-script tokens were stripped. Digits are kept.

Language filter. `langdetect` has **no 'bho' label** – it cannot itself identify Bhojpuri text, and Hindi/Bhojpuri share vocabulary and script closely enough that mislabeled Bhojpuri leaking into Hindi's corpus was a real risk during collection (Bhojpuri was this project's original Model L – see Section 6 for why it was superseded by Nepali; this filter and the correction below predate that switch and remain part of Hindi's own cleaning history, run unconditionally regardless of which language is currently Model L). `dropped_language.likely_bhojpuri` is an active reject filter: any document where Bhojpuri lexical markers (बा/बाटे/बाड़ें/एगो/गइल/भइल/कइल/etc., ≥2 hits) outnumber Hindi markers (है/हैं/एक/गया/हुआ/कयि/etc.) is dropped before reaching `data/cleaned/`. 1,365 documents were dropped by this filter.

Historical cross-language correction (Hindi ↔ Bhojpuri). `scripts/validate_phase1.py`'s cross-language check, run while Bhojpuri was still Model L, caught 2 documents whose content hash was shared between the Hindi and Bhojpuri cleaned corpora – genuinely Bhojpuri news text that `ai4bharat/sangraha` had sourced as `hin`, an upstream labeling error. Both were removed from Hindi's cleaned corpus. This is why Hindi's final cleaned count is **704,101**, not `clean_stats.json`'s own recorded 704,103. This correction is permanent, independent of which language is currently Model L.

Deduplication. Exact-hash matching on normalized text (`dedup_key`); on a manual/downloaded tie, the manual copy wins. Near-duplicate detection beyond exact-hash matching was **not** run on Hindi's pipeline – see Section 8.

Reason	Docs
Malformed Unicode	17
Too short (<200 chars)	63,132
Low Devanagari ratio	228
Likely Bhojpuri (active reject filter)	1,365
langdetect mismatch	1,778
Exact/near-duplicate	2,983
Cross-language duplicate (targeted correction)	2
Total dropped	**69,505**
Docs in → cleaned	773,606 → 704,101

Table 3: Hindi measured drop counts. Too-short documents are the largest single loss (90.8%).

3.2 Nepali cleaning pipeline

Stage-based (`nepali/scripts/s00_doctor.py` through `s14_corpus_stats.py`), sharing the same `corpuslib` code as Hindi's own Nepali-parallel pipeline copy (byte-identical, hashed at CI time, operating on entirely separate data) – not shared with Hindi's `clean_lib.py`.

Language ID. `narrow_margin_vs_hi` (600 of 2,655,504 docs) is the language-ID model catching documents too ambiguous to confidently call Nepali rather than Hindi – the same underlying risk Hindi's `likely_bhojpuri` filter addresses for its neighbor language, handled here via a margin-vs-Hindi check.

Stage	Docs in	Docs out	Retention
S03 normalise	2,655,504	2,655,504	100.00%
S04 language ID	2,655,504	2,649,643	99.78%
S07 train-pool split-off	2,649,643	2,251,378	84.97%
S05 heuristic filters	2,251,378	2,245,354	99.73%
S06 perplexity filter	2,245,354	2,178,418	97.02%
S08 dedup (MinHash/LSH)	2,178,418	2,126,334	97.61%
S09 decontaminate	2,126,334	2,109,223	99.20%
S10 balance lanes (train only)	2,109,223	1,520,000	72.06%

Table 4: Nepali document retention through the pipeline.

Deduplication is MinHash/LSH, not exact-hash (`num_perm=100, bands=20, rows=5, shingle_size=5, threshold=0.55`) – a near-duplicate method Hindi's pipeline does not use. Removed 52,084 of 2,178,418 documents (2,350 exact, 49,734 near); 21,273 cross-lane duplicate pairs, all resolved to the manual copy.

Decontamination (S09) – a step Hindi's pipeline has no equivalent of: 17,111 train documents matching held-out (val+test) content were removed. A rescan afterward found 0 remaining matches.

Lane balancing (S10). Train's raw manual share was 23.27% going into S10. Rather than loosening a filter, S10 trimmed the **downloaded** lane (589,223 docs dropped, manual untouched) to reach 27.42%.

Reason	Docs
wrong_language (S04)	1,599
low_confidence (S04)	1,189
narrow_margin_vs_hi (S04)	600
too_short (S04)	251
other script-ratio buckets, $\leq$5 each (S04)	121
repeated_lines (S05)	1,858
too_few_sentences (S05)	1,226
too_short (S05)	382
link_farm (S05)	481
low_lexical_diversity (S05)	5
too_few_words (S05)	8
mean_word_too_long (S05)	1

Table 5: Nepali measured drop reasons (S04 language-ID + S05 heuristics).

3.3 Cross-language check (current: Hindi vs. Nepali)

Run via `scripts/check_nepali_hindi_overlap.py`: every one of Nepali's 1,918,265 final documents had its `dedup_key` (identical algorithm to `clean_lib.dedup_key`, computed fresh) checked against the full set of 704,101 dedup_keys already computed for Hindi's cleaned corpus. **Result: 0 shared documents.** Full run took 467.7 seconds.

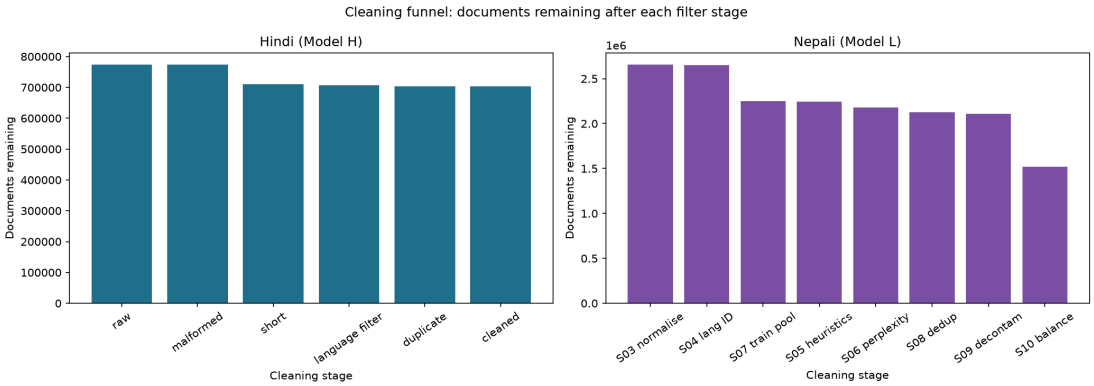


Figure 1: Cleaning funnel per language.

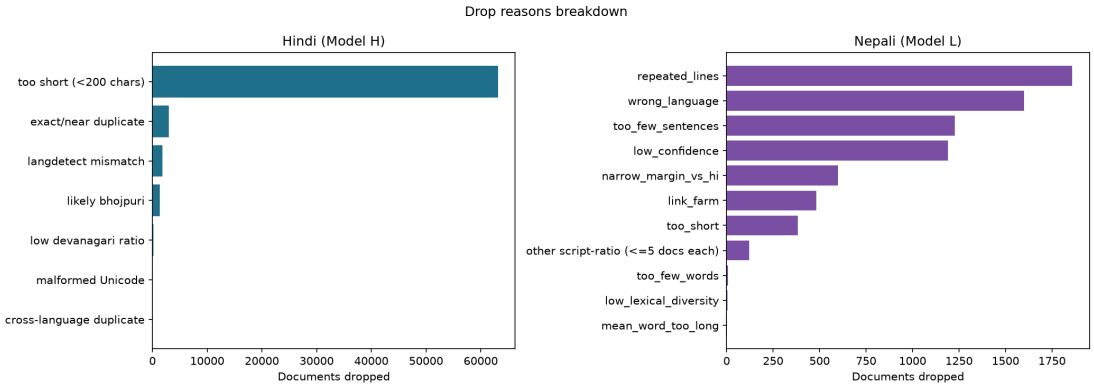


Figure 2: Drop reasons breakdown per language.

4 Splits

Hindi splits at the **document level**, seed 42, using deterministic hashing of `document_id`.

	Hindi (train/val/test)
Documents	675,937 / 14,082 / 14,082
Ratio	∼95.9% / 2.0% / 2.0%

Nepali's split happens earlier, at S07 – val/test are held out from the full document pool *before* S08 (dedup), S09 (decontamination), and S10 (lane balancing) run on what remains, so those later stages never touch documents that will end up in eval.

	Nepali (train/val/test)
Documents	1,520,000 / 199,428 / 198,837
Ratio	∼79.2% / 10.4% / 10.4%

`scripts/validate_phase1.py` reports no overlap for Hindi. `scripts/validate_phase1_nepali.py` and `split_stats.json` (`zero_doc_id_overlap: true`) independently confirm the same for Nepali.

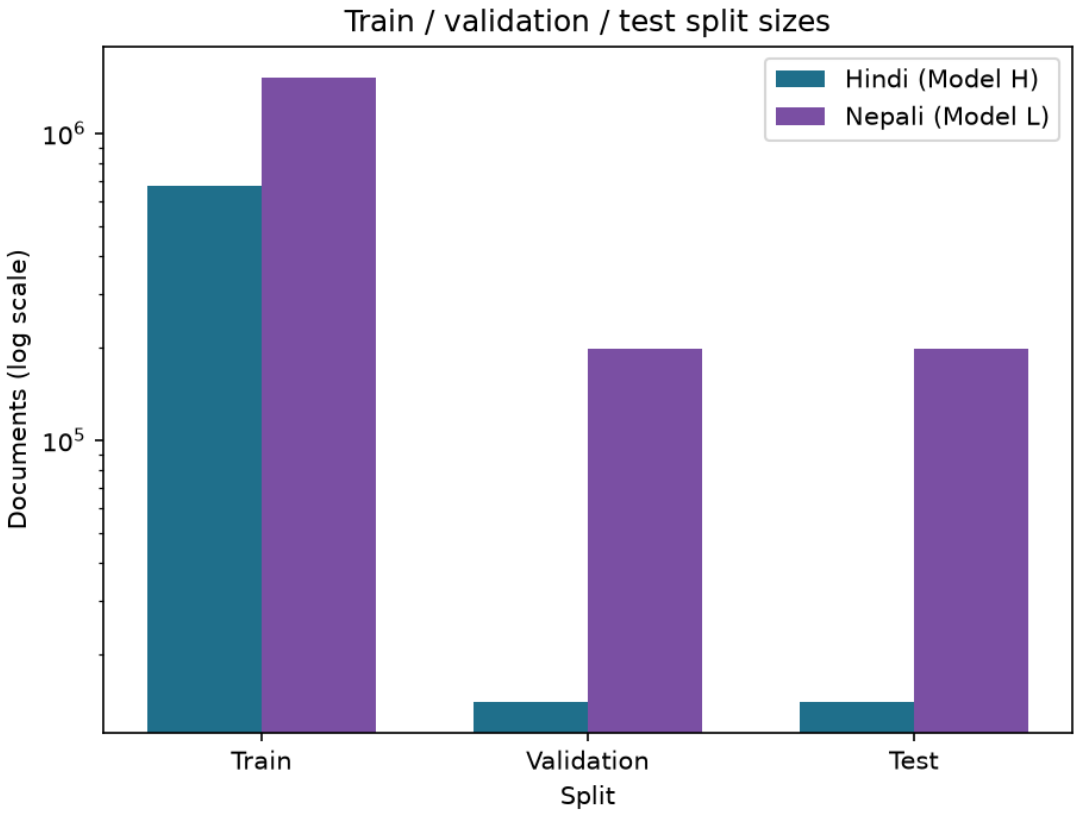


Figure 3: Train/validation/test split sizes, both languages.

5 Tokenizers

5.1 Hindi: SentencePiece Unigram

Trained on Hindi's own train split only. Unigram treats the segmentation itself as a latent variable, re-estimating piece probabilities via EM – argued to suit Hindi's agglutinative-suffixing morphology and Devanagari conjunct clusters better than BPE's greedy, order-dependent merging. This is structural, not empirically confirmed for Hindi specifically (see Section 8) – though the Nepali subsection below reports an actual BPE-vs-Unigram comparison that did run, for the other language. Unigram's suffix-array stage peaked near 5.9GB on this project's 16GB machine.

5.2 Nepali: SentencePiece BPE

`nepali/configs/tokenizer.yaml` configures `model_type: bpe` as this pipeline's default, configured independently of Hindi's Unigram choice. The config includes a genuine safety net: `compare_unigram.enabled: true`, set to also train Unigram and keep whichever scores better *if* BPE's fertility exceeded 2.10 tokens/word. It never triggered – BPE's measured fertility (1.4825–1.6988) stayed well under that threshold.

`byte_fallback` is explicitly **disabled** (`byte_fallback: false`), unlike Hindi's `byte_fallback= True` – a real methodological difference for interpreting UNK rate below.

5.3 Full sweep table

Vocabulary range: **5,000–10,000** for both languages, per a course-TA ruling capping vocab size at this range for a ∼25M-parameter model. Every number below was **re-measured fresh** by `report/phase1/make_plots.py`, loading each trained `.model` file and encoding the first 2,000 non-empty documents of each language's own held-out validation split. Hindi's fresh numbers match `hindi_pipeline_manifest.json` exactly. Nepali's fresh numbers are close to but not bit-identical to its own pipeline's `selection_report.json` (e.g. fertility 1.4825 here vs. 1.4868 there, at vocab=10,000) – both real measurements, on two different same-methodology 2,000-document samples.

Vocab	chars/tok	fertility	UNK	byte-fb	coverage	unused ids
5,000	3.3230	1.4961	0.000000	0.018301	0.9654	173
8,000	3.5622	1.3956	0.000000	0.019613	0.9762	190
10,000	3.6601	1.3583	0.000000	0.020213	0.9767	233

Table 6: Hindi tokenizer sweep.

Vocab	chars/tok	fertility	UNK	byte-fb	coverage	unused ids
5,000	3.7882	1.6988	0.000000	0.000000	0.9776	112
6,000	3.9410	1.6329	0.000000	0.000000	0.9727	164
8,000	4.1725	1.5424	0.000000	0.000000	0.9616	307
10,000	4.3410	1.4825	0.000000	0.000000	0.9458	542

Table 7: Nepali tokenizer sweep.

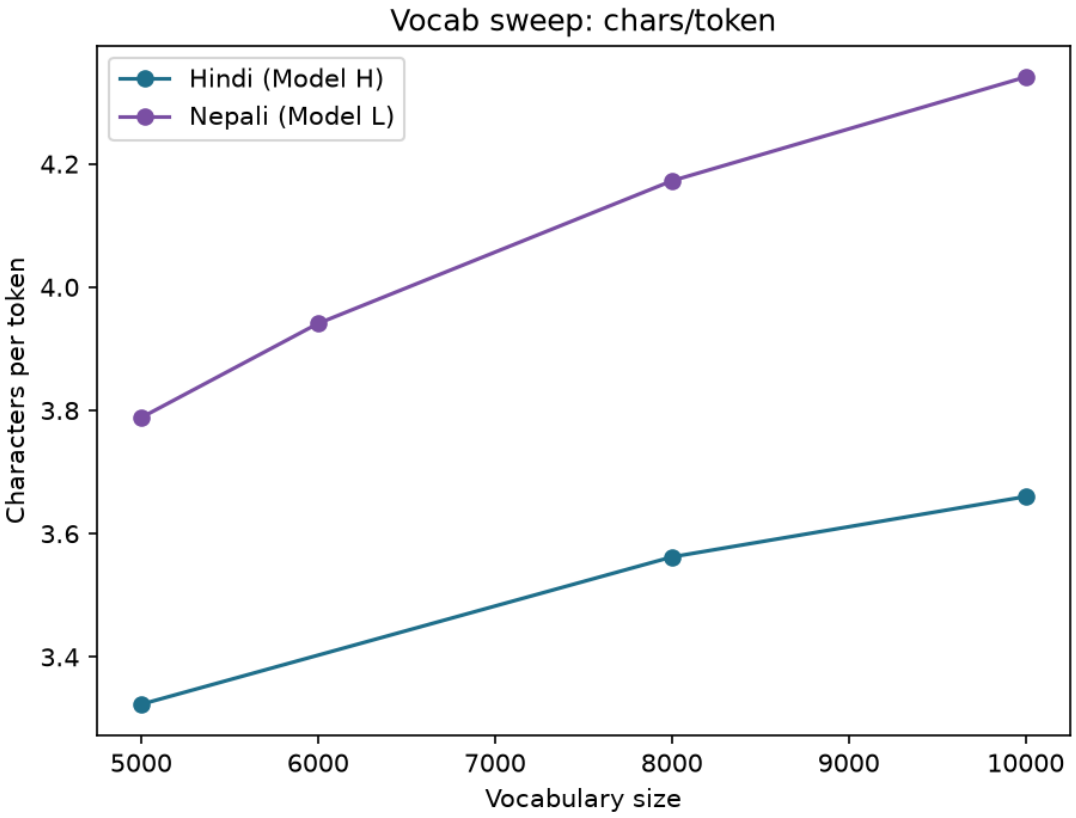


Figure 4: Vocab sweep: characters per token, both languages.

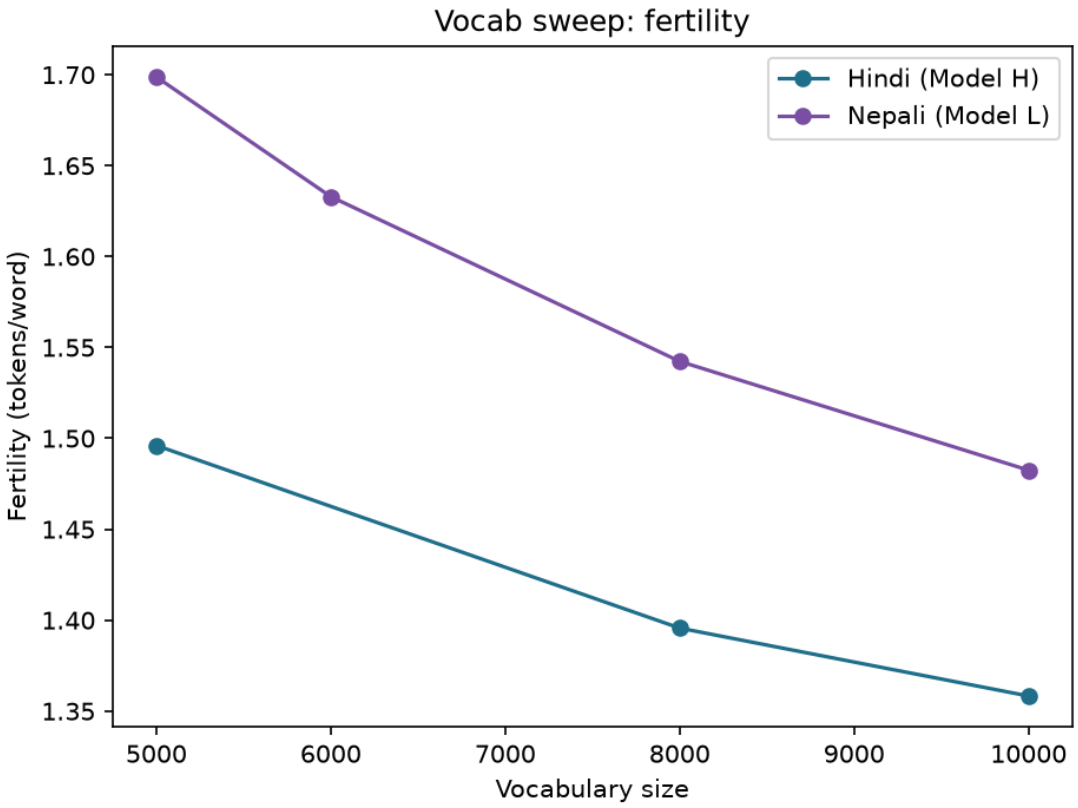


Figure 5: Vocab sweep: fertility (tokens per word), both languages.

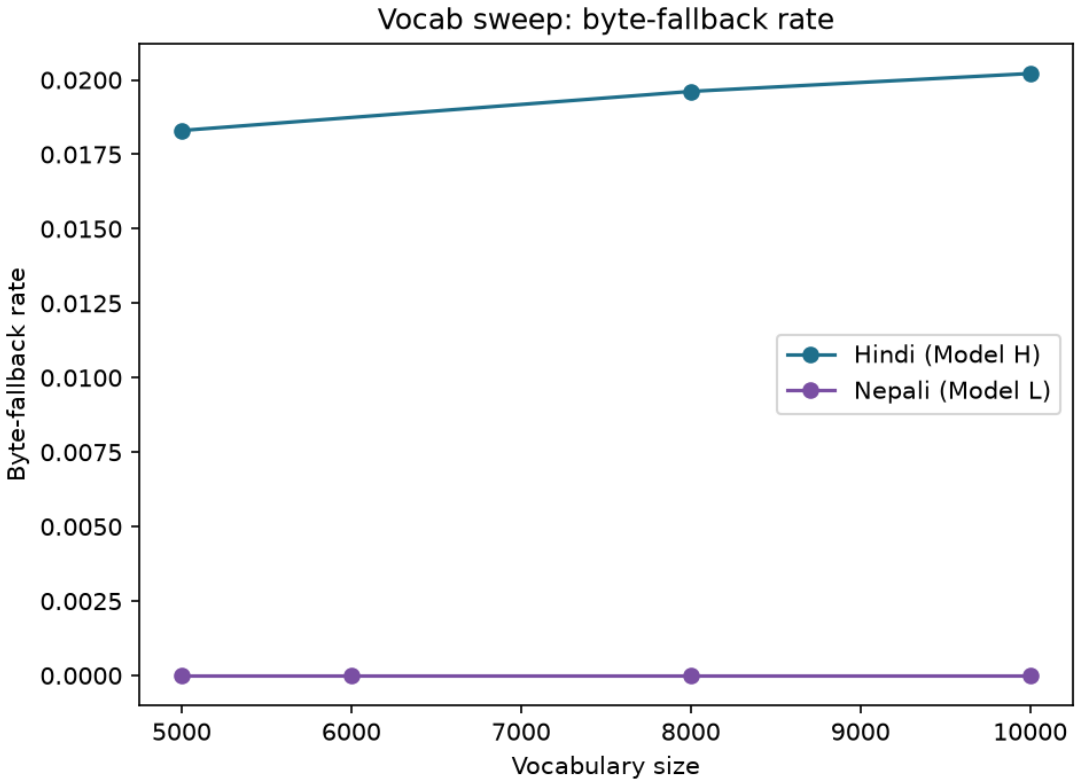


Figure 6: Vocab sweep: byte-fallback rate, both languages.

5.4 Why UNK rate is meaningless for Hindi but real for Nepali

For **Hindi**, UNK rate reads exactly 0.0 at every vocab size, but this is structural: `byte_fallback=True` means any character sequence the model cannot represent normally falls back to raw-byte pieces instead of `<unk>`. The metric that actually tracks real out-of-vocabulary pressure for Hindi is **byte-fallback rate** (1.83%–2.02% across the sweep).

For **Nepali**, `byte_fallback` is disabled, so a 0.0 UNK rate at every vocab size is not masked by a fallback mechanism – it means the trained vocabulary genuinely represented 100% of the validation sample's characters directly, with zero fallback of any kind. This is a real, meaningful difference in what "UNK rate = 0" means between the two languages' tokenizers.

5.5 Vocab-size choice

Hindi: 8,000, chosen on a fertility/embedding-budget trade-off. The fertility gain from 8k→10k (2.7% relative) is much smaller than 5k→8k (6.7% relative), for meaningfully more embedding cost. Measured directly: 8,000 produces more total tokens than 10,000 for Hindi (505,852,028 vs. 491,883,387), pushing Hindi to 101.2% of target versus 10,000's 98.4%.

Vocab	Embed params (d=512)	% of 25M
8,000	4,096,000	16.38%
10,000	5,120,000	20.48%

Table 8: Embedding parameter budget at a ∼25M-param model.

Nepali: 10,000, chosen by the pipeline's own fertility-elbow rule, not a manually argued trade-off: `selection_report.json` records *"no elbow found within the swept range; fertility was still improving by more than 3% at every step, so the largest candidate (10,000) was selected."* At 10,000, Nepali's embedding table is 5,120,000 params (20.48% of a 25M budget) – still within the TA's cap, at the top of the range rather than the middle Hindi landed on.

5.6 Sample tokenizations (real model output)

Hindi (vocab_size=8,000):

text: अतुल माहेश्वरी के नधिन के कई महीनों बाद अमर उजाला में...
pieces: ['_अ', 'तुल', '_माह', 'ेश्वर', 'ी', '_के', '_नधिन', ...]

text: नई दल्लिी
pieces: ['_नई', '_दल्लिी', '<0x0A>', 'नर', 'ेंद्र', '_मोदी', ...]

Nepali (vocab_size=10,000):

text: नेपाली पद वर्ग सङ्खेय नाम अर्थ कमजोर भएको अवस्था; गल्ती
pieces: ['_नेपाली', '_पद', '_वर्ग', '_सङ्', 'खे', 'य', '_नाम', ...] (45 tokens total)

text: नेपाली पद वर्ग नाम अर्थ मान्छे; मानसि
pieces: ['_नेपाली', '_पद', '_वर्ग', '_नाम', '_अर्थ', '_मान्छे', ';', ...] (45 tokens total)

See Figure 7 for how these pieces fit into the whole vocabulary's usage distribution.

Both languages show the expected Zipf-shaped curve, plotted at each language's own selected vocab size (Hindi 8,000, Nepali 10,000 – they differ, unlike Hindi/Bhojpuri which happened to both

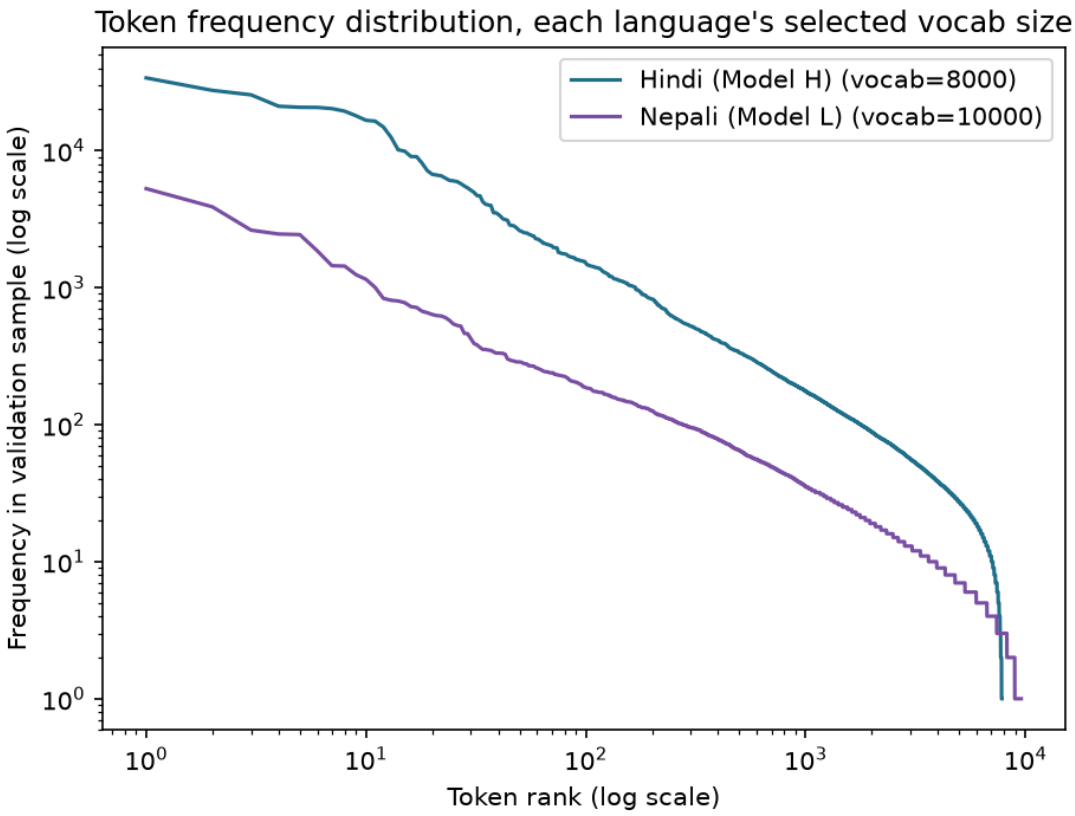


Figure 7: Token frequency distribution (log-log), each language at its own selected vocab size.

choose 8,000).

5.7 Training sample size – stated plainly

SentencePiece does not train on the entire train split. Hindi uses `input_sentence_size=2,000,000`; Nepali's config sets `input_sentence_size=3,000,000`.

Language	Chars used for training	Train split total chars	Fraction
Hindi	∼372.7M	1,692,881,268	22.0%

Nepali's exact sampled-character count was not separately logged by its pipeline the way Hindi's was – a real reporting gap, noted rather than backfilled with an estimate (Section 8).

6 Results

Hindi: 505,852,028 total tokens (train 485,280,741 + validation 10,484,640 + test 10,086,647) – **101.2% of the ∼500M target**. Manual share: **28.65% char-weighted**, **30.66% token-weighted**. Both clear the 20% floor.

Nepali: 635,204,333 total tokens (train 519,906,359 + val 57,839,396 + test 57,458,578) – **127.0% of the ∼500M target**, exceeding it by a wider margin than Hindi. Manual share: **26.41% token-weighted** (Nepali's pipeline never tracked per-lane characters, so no char-weighted figure exists for it). Both of the pipeline's own absolute lane targets are also cleared independently: manual 167,787,251 $\geq$ 100,000,000; downloaded 467,417,082 $\geq$ 400,000,000.

Nepali was not always the plan. This project's original Model L was **Bhojpuri**: fully built, cleaned, split, tokenized, and passing every validation check, with a documented, spec-permitted 16.8%-of-target shortfall (84,217,032 tokens) since Bhojpuri's available public data is genuinely much smaller than Nepali's. Once Nepali's own pipeline cleared both its absolute lane targets, Bhojpuri's tracked pipeline code and tokenizer files were removed from the repository (`bhojpuri/`) – its numbers above are the historical record of that work, not a live, browsable directory. Nepali's pipeline was resumed after a course-wide deadline extension made finishing it worthwhile; `auto_grow_and_tokenize.py` then grew whichever lane fell short of its absolute target and re-ran the full pipeline until both targets were met, landing at 127.0% of the planning target rather than a documented shortfall.

7 Reproducibility

	Hindi	Nepali
Split mechanism	seed 42, hash	S07 holdout, pre-S08–S10
Tokenizer determinism	`shuffle_input_sentence=True`	`shuffle_input_sentence=True`
Python	3.14.0	3.14.0
sentencepiece	0.2.2	0.2.2

To rerun Hindi:

```
python hindi/data/pipeline/download_hindi.py
python hindi/data/pipeline/ingest_manual.py
```

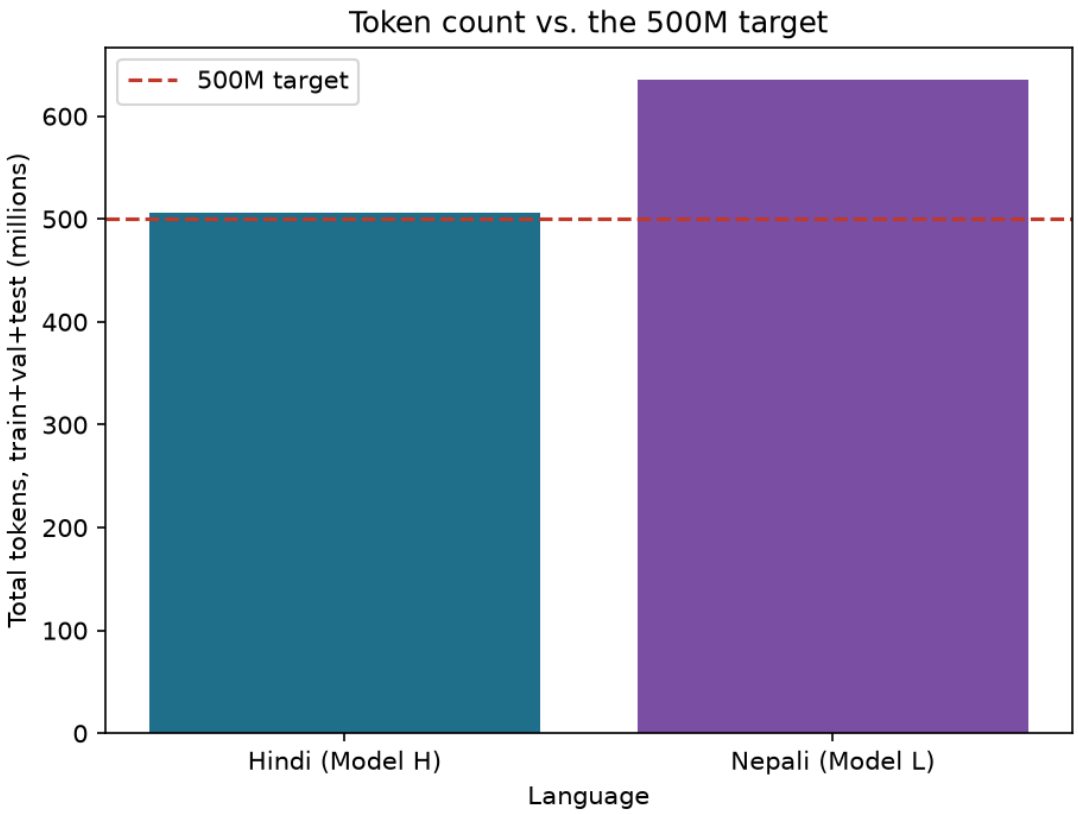


Figure 8: Token count vs. the 500M target, both languages.

```
python hindi/data/pipeline/clean_hindi.py
python hindi/data/pipeline/split_hindi.py
python hindi/tokenizer/train_tokenizer.py --vocab-size 8000
python scripts/validate_phase1.py
```

To rerun Nepali:

```
cd nepali/scripts
python s00_doctor.py
python auto_grow_and_tokenize.py
python ../../scripts/validate_phase1_nepali.py
python ../../scripts/check_nepali_hindi_overlap.py
```

Provenance note (Nepali only): the corpus/tokenizer artifacts this report cites for Nepali were produced by running `nepali/scripts/` on a separate machine (the ∼30+ hour S08 MinHash dedup pass was impractical on the primary machine within the deadline window); only the pipeline's output files were transferred into this repo, not live per-stage console logs. `log.md`'s Nepali S08(production)–S14 entries are reconstructed from those output files, flagged inline in `log.md` itself.

Large artifacts are hosted on OneDrive: https://iiithydstudents-my.sharepoint.com/:f:/g/personal/swaraj_kumar_students_iiit_ac_in/IgCh3u8O2sr4QIJqmrUS1CllAV1C5HL0R9SOWUUVIzTgcdw?e=bgUHaG (an IIIT-institutional link, used deliberately rather than Google Drive).

8 Limitations and What I Would Do Differently

Hindi:

- **The language-ID proxy is a real, acknowledged weak point**, though now largely historical since Bhojpuri is no longer this project's Model L. `langdetect` has no Bhojpuri label, so `likely_bhojpuri` is a coarse lexical-marker filter, not a trained classifier. This filter still runs unconditionally as part of Hindi's own cleaning, even though its original purpose no longer applies day to day.
- **SentencePiece training samples the corpus** (22.0% of train characters). Standard practice; does not bias the held-out evaluation numbers.
- **Near-duplicate detection beyond exact-hash matching was cut for time on Hindi's pipeline specifically.** Nepali's pipeline, by contrast, does run MinHash/LSH – now an asymmetry between the two languages, not a shared gap.
- **The BPE-vs-Unigram argument for Hindi is structural, not empirically confirmed for Hindi specifically.** Nepali's pipeline, by contrast, runs this comparison automatically – it just never triggered.
- **A small raw-total discrepancy** (528 characters, 0.00003%) was not root-caused. Too small to affect any pass/fail check.

Nepali:

- **Manual share is token-weighted, not char-weighted**, unlike Hindi's – Nepali's pipeline never added per-lane character tracking.

- **Manual-lane concentration**: 91.6% of manual tokens come from one source (`news_onlinekhabar`).
- **SentencePiece's sampled-character count was not logged** by Nepali's pipeline the way Hindi's was.
- **The Nepali pipeline ran on a separate machine**; its per-stage console logs for S08–S14 were not captured live, only reconstructed from output artifacts in `log.md`.
- **Cross-language check covers Nepali vs. Hindi only** (Section 3.3), not Nepali vs. Bhojpuri, since Bhojpuri is not part of the final submission.

Sources and References

1. Hindi Wikipedia XML dump (self-extracted via WikiExtractor). Source: Wikimedia dumps, https://dumps.wikimedia.org.
2. Nepali Wikipedia, Wikibooks, Wiktionary XML dumps (self-parsed). Source: Wikimedia dumps, https://dumps.wikimedia.org.
3. Gadya Kosh (Hindi prose wiki). https://gadyakosh.org.
4. Patrika (Hindi newspaper archive). https://www.patrika.com.
5. OnlineKhabar (Nepali news, largest manual source). https://www.onlinekhabar.com.
6. BBC Nepali, Ekantipur, Ratopati, Nagarik (Nepali news sites).
7. Internet Archive (OCR/text extraction). https://archive.org.
8. HuggingFaceFW/fineweb-2 (Hindi partition). https://huggingface.co/datasets/HuggingFaceFW/fineweb-2.
9. AI4Bharat Sangraha (verified and unverified Hindi and Nepali partitions). https://huggingface.co/datasets/ai4bharat/sangraha.
10. AI4Bharat IndicCorpV2 (Nepali partition). https://huggingface.co/datasets/ai4bharat/IndicCorpV2.
11. `wikimedia/wikipedia:20231101.hi` – evaluated, quarantined, not used (redundant with the self-extracted Hindi Wikipedia dump). https://huggingface.co/datasets/wikimedia/wikipedia.
12. SentencePiece (tokenizer library). Kudo, T. and Richardson, J. (2018). *SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing.* EMNLP 2018 (System Demonstrations). https://github.com/google/sentencepiece.
13. `langdetect` (Python language-identification library, used by Hindi's pipeline; known no-Bhojpuri-label limitation documented throughout Section 3). https://pypi.org/project/langdetect/.